Інструкція до практичної роботи №80

ТЕМА: "Створення програм з функціями "

МЕТА: Ознайомитись з рекурсивними функціями мові С++. Набути практичних навичок створення програм з використанням функцій.

Теоретичні відомості

Що таке рекурсія? Що називається рекурсією?

Будь-яка функція (метод) у своєму тілі може викликати сама себе. Рекурсія – це такий спосіб задавання функції, при якому результат повернення з функції для даного значення аргументу визначається на основі результату повернення з тієї ж функції для попереднього (меншого або більшого) значення аргументу.

Якщо функція (метод) викликає сама себе, то такий виклик називається рекурсивний виклик функції. При кожному рекурсивному виклику запам’ятовуються попередні значення внутрішніх локальних змінних та переданих у функцію параметрів. Щоб наступний крок рекурсивного виклику відрізнявся від попереднього, значення як мінімум одного з параметрів функції повинно бути змінене. Припинення процесу рекурсивних викликів функції відбувається, коли змінюваний параметр досягнув деякого кінцевого значення, наприклад, оброблено останній елемент в масиві.

Щоб циклічний процес перетворити в рекурсивний, потрібно вміти визначити (виділити) три важливі моменти:

- умову припинення послідовності рекурсивних викликів функції. Наприклад, якщо лічильник або ітератор з іменем k змінюється від 1 до 10 у зростаючому порядку, то умова припинення є досягнення значення k=10. Умова припинення вказується в операторі return;
- формулу наступного елементу або ітератору, що використовується в рекурсивному процесі. Ця формула вказується в операторі return;
- перелік параметрів, що передаються в рекурсивну функцію. Один з параметрів обов’язково є ітератор (лічильник), який змінює своє значення. Інші параметри є додатковими, наприклад, посилання на масив, над яким здійснюється обробка.

Розгляньте приклади за подібним зразком яких можна створювати власні рекурсивні функції.

Приклад 1. Створити програму знаходження факторіалу числа – n!

Обчислення факторіалу числа методом рекурсії є майже в кожному підручнику. Факторіал числа обчислюється за формулою

$f = 1 \cdot 2 \cdot \ldots \cdot (n-1) \cdot n$ де $n>=1$.

```cpp
#include <iostream>

using namespace std;

int fact(int n){
    if (n==1) return 1;
    else  return fact(n-1)*n;
}

int main(){
    int n;
    cin>>n;
    cout<<fact(n);
    return 0;
}
```

Приклад 2. Створити програму знаходження найбільшого спільного дільника двох натуральних чисел (алгоритм Евкліда)

Відомо, що найбільший спільний дільник (НСД) визначається за формулою Евкліда:

```
// найбільший спільний дільник за алгоритмом Евкліда
int MaxDiv(int n, int m)
{
   if (n==m)
      return n;
   else
   if (n>m)
      return MaxDiv(n-m, m);
   else
      return MaxDiv(n, m-n);
}
```

Програмний код програми з використанням рекурсивної функції наступний:

```
#include <iostream>

using namespace std;

int MaxDiv(int n, int m)
{
    if (n==m)
        return n;
    else
    if (n>m)
        return MaxDiv(n-m, m);
    else
        return MaxDiv(n, m-n);
}

int main()
{
      int n,m;
   cin>>n>>m;
    cout<<"NSD=" << MaxDiv( n, m);
    return 0;
}
```

Переваги та недоліки використання рекурсії

Рекурсію часто порівнюють з ітерацією. Організація циклічного процесу з допомогою рекурсії має свої переваги та недоліки.

Можна виділити такі взаємозв‘язані переваги рекурсії:

- природність (натуральність) представлення складних, на перший погляд, алгоритмів;
- рекурсивний алгоритм більш читабельний у порівнянні з ітераційним;
- для багатьох поширених задач рекурсію легше реалізувати ніж ітерацію.

Рекурсія добре підходить для реалізації алгоритмів обходу списків, дерев, аналізаторів виразів, комбінаторних задач тощо.

Недоліки рекурсії полягають у наступному:

- порівняно з ітерацією багаторазовий виклик рекурсивної функції потребує більше часу. Це зв‘язано з тим, що при виклику рекурсивного методу його параметри копіюються в стек. При завершенні виклику рекурсивної функції попередні значення параметрів витягуються зі стеку, що призводить до зайвих операцій. Ітераційний алгоритм для тієї самої задачі працює швидше;
- якщо рекурсивна функція містить великі об’єми локальних внутрішніх змінних і велику кількість параметрів, то використання рекурсії не є ефективним. Це зв’язано з тим, що для кожного рекурсивного виклику потрібно робити копії цих змінних та параметрів. При великій кількості рекурсивних викликів це призведе до надмірного використання пам’яті.

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані залежать від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
2	n = N + 3, m = N mod 4 + 1	n = 10, m = 4

б) Кожне завдання — окремий файл Прізвище_ПР80_зN.cpp (наприклад, Ivanenko_PR80_z2.cpp), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Компілюйте й запускайте програми в Code::Blocks, Dev-C++, Visual Studio чи іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот вікна виконання кожної програми з введеними даними.

г) Підключайте <iostream> (за потреби <cmath>, <iomanip>) і using namespace std; заголовки <iostream.h> і <conio.h> із прикладів застаріли — замість getch() використовуйте cin.get() або system("pause"). Якщо кирилиця в консолі спотворюється, додайте SetConsoleCP(1251); SetConsoleOutputCP(1251); (<windows.h>). Тексти виводу — точно як у прикладах. Для факторіала використовуйте тип long long і обмеження n ≤ 20; функцію fact із теорії виправте так, щоб fact(0) = 1 (інакше при n = 0 виникає нескінченна рекурсія).

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

Хід роботи

1. Набрати приклади 1 та 2 приведені в теоретичних відомостях.

Приклад: fact: 5 → «120»; MaxDiv (НСД): 48 18 → «NSD=6».

2.Створити програму для обчислення кількості комбінацій з *n* різних елементів по *m* використавши функцію для обчислення факторіалу.

Приклад (N = 7): n = 10, m = 4 → «C(10,4) = 210»; n = 5, m = 0 → «C(5,0) = 1»; n = 3, m = 5 → «C(3,5) = 0»; n < 0, m < 0 або n > 20 → «Некоректні дані». Додатково обчисліть C(n, m) рекурсивною функцією з примітки й виведіть «Результати збігаються» або «Результати відрізняються».

$$C_n^k = \frac{n!}{k!\,(n-k)!}.$$

Увага!

Кількість комбінацій можна визначати рекурентною формулою

$$C_n^m = \begin{cases} 1, \text{ якщо } m = 0, n > 0 \text{ або } m = n \geq 0; \\ 0, \text{ якщо } m > n \geq 0; \\ C_{n-1}^{m-1} + C_{n-1}^m \text{ в інших випадках.} \end{cases}$$

Примітка. Рекурсивна функція обчислення кількості комбінацій з n різних елементів по m

```
// кількість комбінацій з n по m
int C(int n, int m)
{
    if ((m==0)&&(n>0) || (m==n)&&(n>0))
        return 1;
    else
        if ((m>n)&&(n>=0))
            return 0;
        else
            return C(n-1,m-1) + C(n-1,m);
}
```

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Завдання 1	3 бали	3
Завдання 2	8 балів	8
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (не виконано вимогу умови, не оброблено m > n чи n = 0, неправильний формат результату) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити n і m чи тип результату). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

1. Що таке рекурсія? Із яких частин складається рекурсивна функція?
2. Що таке базовий випадок рекурсії і що буде, якщо його пропустити?
3. Як записати формулу кількості комбінацій через факторіали?
4. Чим рекурсивне обчислення факторіала відрізняється від циклічного?

Відповідайте на контрольні питання письмово у звіті (документ Word разом зі скріншотами): 1–2 речення на кожне питання, власними словами.

Література

1. Глинський Я.М., Анохін В.Є., Ряжська В.А. C++ і C++ Builder. Навчальний посібник. – Львів: СПД Глинський, 2006.
2. Глушаков С.В., Смирнов С.В., Коваль А.В. Практикум по C++. – Харьков: Фоліо, 2006.
3. Лєхан С.Ф. Інформатика C++. Спецкурс. 10-12 класи. – Шепетівка; Аспект, 2007.
4. https://www.bestprog.net/uk/2019/01/13/examples-of-tasks-solving-on-recursion-in-the-c-programming-language-ua/#q01